

+Introducción a Bash

¡Aproveche el poder de la línea de comandos!

Agenda

- Teoría
 - ¿Qué es una interfaz de línea de comandos?
 - ¿Qué es una shell?
 - ¿Qué es una Terminal?
 - ¿Qué es un script de Shell?
 - ¿Qué es Bash?
- Fundamentos de bash:
 - sintaxis
 - obtener ayuda con los comandos
 - Comodines
 - Comandos fundamentales
 - Redirección y Pipelines
- Ejercicio práctico
 - Terminar un proceso
 - Salida de filtro con grep
 - Un simple script de bash
 - Instalar paquetes
 - Manejar entornos

¿Qué es una interfaz de línea de comandos?

Un programa basado en texto que realiza acciones a través de comandos (por ejemplo, CLI de Azure)

- CLI de Azure
- CLI de Docker

¿Qué es una shell?

Un shell es un programa que ejecuta comandos (desde CLI o integrados) y genera resultados. Es una de las partes más importantes de un sistema Unix. Hay muchos shells de Unix diferentes, pero todos derivan características del Bourne Shell `/bin/sh`

¿Qué es una Terminal?

Un programa GUI utilizado para ejecutar un shell

- iTerm2

- Guake -Terminal de Windows

¿Qué es un script de shell?

Un archivo de texto que contiene una secuencia de comandos de shell.

¿Qué es Bash?

Linux usa una versión mejorada de Bourne Shell llamada Bash. Bash significa Bourne Again Shell y se ha convertido en el estándar de facto para administrar máquinas Linux.

Fundamentos de bash

La sintaxis completa para un comando Bash es
`comando [opciones] [argumentos]`

- El primer elemento es el comando.
- Los comandos suelen ir seguidos de argumentos
- La mayoría de los comandos tienen opciones (a menudo llamadas banderas) que modifican su funcionamiento.

```
ls -a /etc
```

- `ls` es el comando de contenido de la lista
- `-a` es una opción (bandera) para incluir archivos/directorios ocultos
- `/etc` es un argumento para el comando `ls` que indica qué directorio listar.

Las banderas se pueden combinar para ser más concisas.

```
# Combinado
```

```
ls -al /etc
```

```
# Forma larga
```

```
ls -a -l /etc
```

Consigue ayuda

No se espera que memorice comandos, argumentos o banderas. Bash tiene documentación integrada en el sistema operativo. Para acceder a él, escribimos `man` (abreviatura de página de manual) seguido del comando.

```
man mkdir
```

También podemos usar el indicador `--help` y nos proporcionará una descripción de la sintaxis y las opciones del comando de manera más concisa que `man`.

```
mkdir --ayuda
```

Comodines (metacaracteres)

Los comodines son símbolos que representan uno o más caracteres en los comandos Bash.

- `*` representa cero o una secuencia de caracteres.
 - Lista de archivos que terminan con `.png` = `ls *.png`
 - Lista de archivos que terminan en `.jpeg` o `.jpg`
`ls *.jpg`
- `?` representa un solo carácter
 - Lista de archivos que comienzan con `000` y terminan con `.jpg` = `ls 000?.jpg`
- Los corchetes `[]` se utilizan para indicar grupos de caracteres.
 - # Lista todos los archivos en el directorio actual que contienen un punto seguido inmediatamente por una `J` o `P` minúscula
`ls *. [jp]*`
- Los nombres de archivos y los comandos distinguen entre mayúsculas y minúsculas.
 - # Lista todos los archivos en el directorio actual que contienen un punto seguido inmediatamente por una `J` o `P` minúscula
`ls *. [jpJP]*`
- Las expresiones entre corchetes pueden representar un rango de caracteres
 - # Lista todos los archivos en el directorio actual que comienzan con una letra minúscula
`ls [a-z]*`
 - # Listar todos los archivos en el directorio actual que están en minúsculas o mayúsculas
`ls [a-zA-Z]*`

Comandos para explorar los conceptos básicos

- `ls`
 - Muestra el contenido del directorio actual o del directorio especificado.
 - Algunos argumentos populares
 - `-a` Incluye archivos y directorios ocultos (los nombres comienzan con un punto)
 - `-l` Incluye más información sobre cada archivo.
- `pwd`
 - imprimir directorio de trabajo.
- `cat`
 - Concatenar archivos a la salida estándar. Manera elegante de decir mostrar el contenido de un archivo en la pantalla.
- `man`
 - Obtener documentación sobre un comando.
- `cd`
 - Significa cambio de directorio.
 - Obtén ayuda con `man builtins`

- Sube con ..
 - Ir a casa con ~
- **mkdir**
 - Crear un directorio.
 - -p para crear un subdirectorio
- **rmdir**
 - Eliminar un directorio vacío.
- **rm**
 - Eliminar un archivo o todos los archivos con *
 - Acostúmbrese a usar -i para que se le pregunte antes de cada eliminación.
- **cp**
 - Copiar archivos y directorios.
 - Este comando reemplazará, así que use -i para que se le solicite antes de sobrescribir.
 - Usar para copiar subdirectorios. Significa recursivo.
- **ps**
 - Obtenga una instantánea de todos los procesos que se están ejecutando actualmente.
- **sudo**
 - Algunos comandos sólo los puede ejecutar el usuario raíz (también conocido como superusuario). No desea iniciar sesión como usuario raíz porque
 - No tiene registro de los comandos que alteran el sistema o quién los está ejecutando.
 - No tiene acceso a su entorno de shell normal.
 - La mayoría de las distribuciones usan un paquete sudo que permite ejecutar comandos como root cuando inician sesión como ellos mismos.
 - Para ejecutar comandos que requieren privilegios de administrador sin iniciar sesión como usuario raíz, anteponga el comando con sudo

Redirección

La redirección de E/S nos permite redirigir la entrada y salida de comandos hacia y desde archivos, así como conectar varios comandos en canalizaciones.

- < para redirigir la entrada a una fuente que no sea el teclado
- > para redirigir la salida a un destino que no sea la pantalla
- >> para hacer lo mismo, pero agregando en lugar de sobrescribir
- | para canalizar la salida de un comando a la entrada de otro

Terminar un proceso de mal comportamiento

1. Cree un programa llamado bad.py que se comportará mal.
Usaremos nano para crear un programa en python. Es el editor de texto universal de Linux. Otros incluyen vim emacs

```
# Crea el programa con vi
nano mal.py
```

```
# Inserta lo siguiente
```

```
yo = 0
while yo == 0:
    pass
```

2. Inicie el programa `bash python3 bad.py &`. El `&` le dice a bash que ejecute el comando en segundo plano y no espere a que termine.

3. Hagamos una lista de todos los procesos que contienen "python"

```
ps -ef | grep python
```

- `ps` enumera los procesos en ejecución en el shell actual
- `-e` muestra todos los procesos
- Listas `-f` en listado de formato completo.

Podemos ver que `bad.py` está consumiendo mucho tiempo de CPU.

4. Usemos el comando `kill` para detener el proceso en ejecución. Primero enumeremos los tipos de señales.

```
# Mostrar una lista de tipos de señales
kill -l
```

- Si quisiéramos reiniciar un proceso, usaríamos `SIGHUP`
- En este caso queremos detener el proceso sin reiniciar, entonces usamos `SIGKILL`

9

5. Tenemos que decidir qué señal vamos a enviar para `kill`. El valor predeterminado es `TERM` (permite que el proceso se cierre de forma segura).

```
kill -9 PROCESS_ID
```

Use Bash y grep para filtrar la salida CLI

Usemos el programa universal de coincidencia de patrones de Linux `grep` para filtrar la salida de la CLI de Azure.

```
cat file.txt | grep text
```

El comando grep

`grep` significa impresión de expresión regular global y es el comando de manipulación de texto más utilizado.

El comando `grep` busca en los archivos de texto texto que coincida con una expresión regular específica y genera cualquier línea que contenga una coincidencia en la salida estándar.

Dos de las opciones más importantes para `grep` son:

- `-i` para coincidencias que no distinguen entre mayúsculas y minúsculas.
- `-v` invierte la búsqueda e imprime líneas que no coinciden.

Las expresiones regulares son un tema completo, pero las tres cosas importantes para recordar son:

- `.` coincide con cualquier número de caracteres, incluido ninguno.
- `+` coincide con uno o más caracteres.
- `.` coincide exactamente con un carácter arbitrario

Juguemos con `grep`. Enumeraremos el contenido de algunos directorios para crear archivos de texto.

```
# /bin directorio que contiene binarios
ls /bin > lista.txt

# /sbin director que contiene binarios con privilegios de superusuario
requeridos
ls /sbin > lista-sbin.txt

# /usr/bin directorio que contiene binarios generales de todo el
sistema.
ls /usr/bin > lista-usr.txt

# /usr/sbin directorio que contiene archivos binarios generales de
todo el sistema con privilegios de superusuario.
ls /usr/sbin > lista-usr-sbin.txt
```

Guión bash simple

1. Escriba un script llamado `create_files` con vim `create_files` con este contenido:

```
#!/bin/bash
# Saluda y crea algunos archivos

echo '¡Creando archivos para ti!'
for f in {a..z} {0..5}; do
    echo hola > "$f.txt"
done
```
2. Haga que el script sea ejecutable con `chmod`
 - `chmod 755` para scripts que todos pueden ejecutar.
 - `chmod 700` para scripts que solo el propietario puede ejecutar.En nuestro caso: `chmod 755 create_files`
3. Ejecute el script `./create_files`
4. Si quisiéramos ejecutar este comando sin `./`, tendríamos que agregarlo a un directorio en nuestro `$PATH`
5. `echo $PATH`
6. Una ubicación ideal para scripts personales es `/usr/local/bin`
7. Mueva el archivo allí `sudo cp create_files /usr/local/bin`
8. Ahora puedes ejecutar `create_files`